



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL RURAL DO RIO DE JANEIRO
CAMPUS NOVA IGUAÇU

ECHOBUND

NOVA IGUAÇU, RJ
2026



UNIVERSIDADE FEDERAL RURAL DO RIO DE JANEIRO- *CAMPUS* NOVA IGUAÇU

Pablo Torres Santos da Silva, Gustavo Luiz da Silva Procópio, Pedro Lucas de Oliveira Alves

ECHOBOUND

Trabalho desenvolvido e apresentado na disciplina de Programação Estruturada, no Curso de Ciência da Computação na Universidade Federal Rural Do Rio de Janeiro – *campus* Nova Iguaçu como avaliação no período.

Professor da disciplina: Daniel Fabio D. Posner

NOVA IGUAÇU, RJ
2026

Sumário

1. Introdução	5
1.1. Tema	5
1.2. Objetivo do projeto.....	5
1.3. Organização do documento.....	5
1.4. Glossário de termos	5
2. Descrição Geral do Sistema	7
2.1. Descrição do Projeto	7
2.2. Principais envolvidos e suas características.....	7
2.2.1. Usuários do Sistema	7
2.2.2. Desenvolvedores do Sistema.....	7
2.3. Regras de Negócio	7
2.3.1. Restrições de negócio.....	7
2.3.2. Restrições de desempenho	8
2.3.3. Tolerância a falhas	8
2.3.4. Ferramentas de apoio.....	8
3. Descrição dos cenários	9
3.1. Requisitos Funcionais.....	9
3.1.1. Caso de Uso: Iniciar Jogo	9
3.1.2. Caso de Uso: Jogar Capítulo	10
3.1.3. Caso de Uso: Entrar em combate	10
3.1.4. Caso de Uso: Avançar Capítulo	11
3.1.5. Caso de Uso: Finalizar Jogo.....	11
3.1. Requisitos Não-Funcionais.....	12
4. Diagramas do Sistema	13
4.1. Diagrama de Caso de uso.....	13
4.2. Diagrama de Classe/Structs	14
5. Desenvolvimento da aplicação	17
5.1. Métodos	17
5.1.1. Modelagem do sistema	17
5.1.2. Implementação.....	17
5.1.3. Testes.....	17
5.1.4. Controle de versão e colaboração	18
5.1.5. Outros	18
5.2. ECHOBOUND	18

5.3.	Jogabilidade	27
	Nessa seção será descrito as teclas e comandos usados para interação com o jogo.	27
5.3.1.	Comandos de navegação no menu principal.....	27
5.3.2.	Comandos na tela de criação de personagem	27
5.3.3.	Comandos de esquiva e controle no combate	27
5.3.4.	Comandos de seleção de diálogo e escolha nos menus.....	27
6.	Conclusão	28
6.1.	Limitações	28
6.2.	Perspectiva Futura	28
6.2.1.	Incremento nos capítulos e na narrativa:.....	28
6.2.2.	Manutenção e Evolução do Código:.....	28
6.2.3.	Melhorias de desempenho e multithreading:.....	28
7.	REFERÊNCIA BIBLIOGRÁFICA E SITES UTILIZADOS	30
	ANEXO - ENTREGA DE CÓDIGOS.....	31

Sumário de Ilustrações

Figura 1: Caso de uso.....	9
Figura 2: Caso de uso.....	13
Figura 3: Diagrama de classes/structs	14
Figura 4: Tela do Menu Inicial	18
Figura 5: Tela Inicial (Créditos)	19
Figura 6: Tela Cutscene Inicial	19
Figura 7: Tela de Criação de Personagem (masculino)	20
Figura 8: Tela de Criação de Personagem (feminino)	20
Figura 9: Introdução Textual	21
Figura 10: Tela Apresentação de Boss 1.....	21
Figura 11: Tela de Batalha 1	22
Figura 12: Menu de Loot	22
Figura 13: Tela de Descanso	23
Figura 14: Tela de Combate 2.....	23
Figura 15: Tela Apresentação Boss 2.....	24
Figura 16: Tela de Combate 3.....	24
Figura 17: Tela de Combate 4.....	25
Figura 18: Tela de Morte	25
Figura 19: Tela de Combate 5.....	26
Figura 20: Escolha Final	26

1. Introdução

O objetivo desta seção é apresentar o projeto, especificando o tema, os objetivos e justificativa para a criação deste projeto e a organização a partir da qual este documento é estruturado.

1.1. Tema

Desenvolvimento de um livro-jogo na linguagem C, com história original escrita pelos discentes. O combate foi projetado de modo a se assemelhar ao jogo *Undertale*, aumentando a interação do jogador com o sistema.

1.2. Objetivo do projeto

Pode-se pensar no objetivo deste trabalho o desenvolvimento da criatividade, das habilidades de trabalho em equipe e arquitetura e projeto de software e sistemas dos discentes, assim como complementar o conteúdo abordado em aula, de forma a possibilitar o uso deste conhecimento no desenvolvimento de um sistema completo.

1.3. Organização do documento

O Capítulo 2 faz uma breve introdução da temática do jogo desenvolvido. Além disso, o Capítulo também apresenta as regras de negócio, envolvendo as restrições de negócio e de desempenho e as ferramentas de apoio. O Capítulo 3 apresenta os requisitos e descrição de cenário do projeto. Os diagramas são mostrados, descritos e discutidos no Capítulo 4. O Capítulo 5 aponta as tecnologias, métodos do desenvolvimento da aplicação, e mostra a aplicação em funcionamento. Por fim, o Capítulo 6 aponta as considerações finais acerca deste projeto e os trabalhos futuros.

1.4. Glossário de termos

- **Livro-jogo:** Gênero de jogo em que o jogador acompanha uma narrativa e toma decisões que influenciam sua progressão e os acontecimentos da história.
- **Capítulo:** Unidade da narrativa que representa uma etapa da aventura, contendo diálogos, exploração e um combate contra um inimigo específico.
- **Andar:** Área da torre correspondente ao capítulo atual, onde ocorrem os eventos e desafios da história.
- **NPC (Non-Player Character):** Personagem controlado pelo sistema, utilizado para diálogos, fornecimento de informações e desenvolvimento da narrativa.
- **Boss:** Inimigo principal de um capítulo, responsável pelo combate que deve ser concluído para que o jogador avance na história.
- **Karma:** Atributo que registra as escolhas do jogador, especialmente relacionadas a matar ou poupar inimigos, influenciando eventos e o desfecho da narrativa.
- **Inventário:** Estrutura responsável por armazenar armas, armaduras e itens consumíveis utilizados pelo jogador durante a partida.

- **Consumível:** Item que pode ser utilizado durante o jogo para recuperar vida ou produzir outros efeitos temporários.
- **Armadura:** Equipamento defensivo composto por diferentes peças que concedem bônus aos atributos do personagem.
- **Persistência de Dados:** Processo de armazenamento das informações do jogador em arquivos binários, permitindo continuar a partida posteriormente.
- **Arquivo Binário:** Arquivo utilizado para salvar e carregar o progresso do jogador, armazenando os dados em formato binário.
- **Struct:** Estrutura de dados da linguagem C utilizada para agrupar informações relacionadas a uma entidade, como jogador, inimigo ou inventário.
- **Módulo:** Conjunto de arquivos responsáveis por uma parte específica do sistema, organizando funcionalidades como combate, narrativa e gerenciamento de objetos.
- **ncurses:** Biblioteca da linguagem C utilizada para criar interfaces em modo texto e controlar elementos visuais diretamente no terminal.
- **Thread Única (Single Thread):** Modelo de execução no qual todas as operações da aplicação são processadas sequencialmente por um único fluxo de execução.
- **Renderização:** Processo de atualização dos elementos visuais exibidos ao jogador na interface do terminal.
- **Colisão:** Verificação utilizada pelo sistema para identificar o contato entre ataques, personagem e inimigos durante o combate.
- **Sprite:** Representação gráfica de personagens, ataques ou objetos, exibida por meio de caracteres ASCII no terminal.

2. Descrição Geral do Sistema

Nesta seção são apresentados os objetivos da aplicação, seu escopo, os envolvidos no projeto e as principais restrições identificadas durante o desenvolvimento.

2.1. Descrição do Projeto

O projeto consiste no desenvolvimento de um livro-jogo digital executado em ambiente de terminal, no qual o jogador acompanha uma narrativa dividida em capítulos e toma decisões que influenciam sua progressão. Durante a jornada, o usuário enfrenta inimigos em um sistema de combate em tempo real, coleta equipamentos, gerencia seu inventário e pode salvar seu progresso para continuar a aventura posteriormente.

A aplicação foi desenvolvida em linguagem C, utilizando a biblioteca **ncurses** para construção da interface em modo texto e arquivos binários para persistência dos dados do jogador. O projeto busca aplicar os conceitos estudados na disciplina de Programação Estruturada, tais utilização de estruturas, manipulação de arquivos, alocação dinâmica de memória em conjunto com conteúdo ainda não trabalhados como organização e modularização do código e criação de diagramas de sistemas.

2.2. Principais envolvidos e suas características

2.2.1. Usuários do Sistema

- Jogadores: usuários que executam o jogo com o objetivo de entretenimento, explorando a narrativa, enfrentando inimigos, coletando equipamentos e progredindo pelos diferentes capítulos da história.

2.2.2. Desenvolvedores do Sistema

Os desenvolvedores do sistema foram os discentes Pablo Torres Santos da Silva, Gustavo Luiz da Silva Procópio e Pedro Lucas de Oliveira Alves, responsáveis pelo levantamento de requisitos, desenvolvimento da aplicação, implementação das mecânicas do jogo, criação da narrativa, elaboração dos diagramas, seleção e criação de sprites e trilha sonora, testes, documentação e manutenção do projeto durante seu desenvolvimento.

2.3. Regras de Negócio

As principais regras e restrições consideradas durante o desenvolvimento são apresentadas a seguir.

2.3.1. Restrições de negócio

1. A reprodução de música só pode ser realizada em sistemas Linux nativos devido ao uso da ferramenta *aplay*, limitando esta funcionalidade a esse ambiente.
2. O progresso do jogador somente pode ser preservado por meio do sistema de salvamento implementado em arquivos binários.

3. A progressão da narrativa ocorre de forma linear entre os capítulos, sendo necessário concluir um capítulo para desbloquear o seguinte.

2.3.2. Restrições de desempenho

1. A movimentação diagonal sofre atrasos em comparação com a movimentação vertical e diagonal devido a execução em thread única e limitações de uso de coordenadas inteiras na matriz.
2. A renderização da interface, leitura de entrada do usuário e atualização da lógica do jogo são executadas sequencialmente, o que pode ocasionar variações de fluidez e de cálculos de colisão.

2.3.3. Tolerância a falhas

1. Falhas de alocação dinâmica de memória podem comprometer a execução da aplicação, uma vez que a linguagem C não dispõe de mecanismos nativos de tratamento de exceções.
2. Arquivos de salvamento corrompidos ou incompatíveis podem impedir o carregamento correto do progresso do jogador.
3. A ausência da biblioteca **ncurses** ou da ferramenta **aplay** impede a utilização completa da aplicação.

2.3.4. Ferramentas de apoio

- **Docker**, utilizado para criação de ambientes isolados e reprodutíveis para o desenvolvimento e para os testes.
- **Git** e **Github**, utilizados para controle de versão e colaboração entre desenvolvedores.
- **GCC**, utilizado como compilador da aplicação.
- **Astah**, utilizado para elaboração dos diagramas UML presentes na documentação do projeto.

3. Descrição dos cenários

Esta seção tem como objetivo descrever os requisitos do sistema.

3.1. Requisitos Funcionais

Neste item apresenta os requisitos funcionais que especificam as ações que um sistema deve ser capaz de executar, ou seja, é uma necessidade funcional (uma função) que o software deve atender. Os requisitos funcionais geralmente são descritos de melhor forma em diagramas de Caso de Uso, juntamente com o detalhamento dos atores e de cada caso de uso. Neste caso, a Figura 1 apresenta o diagrama utilizado para esta descrição e detalhamento.

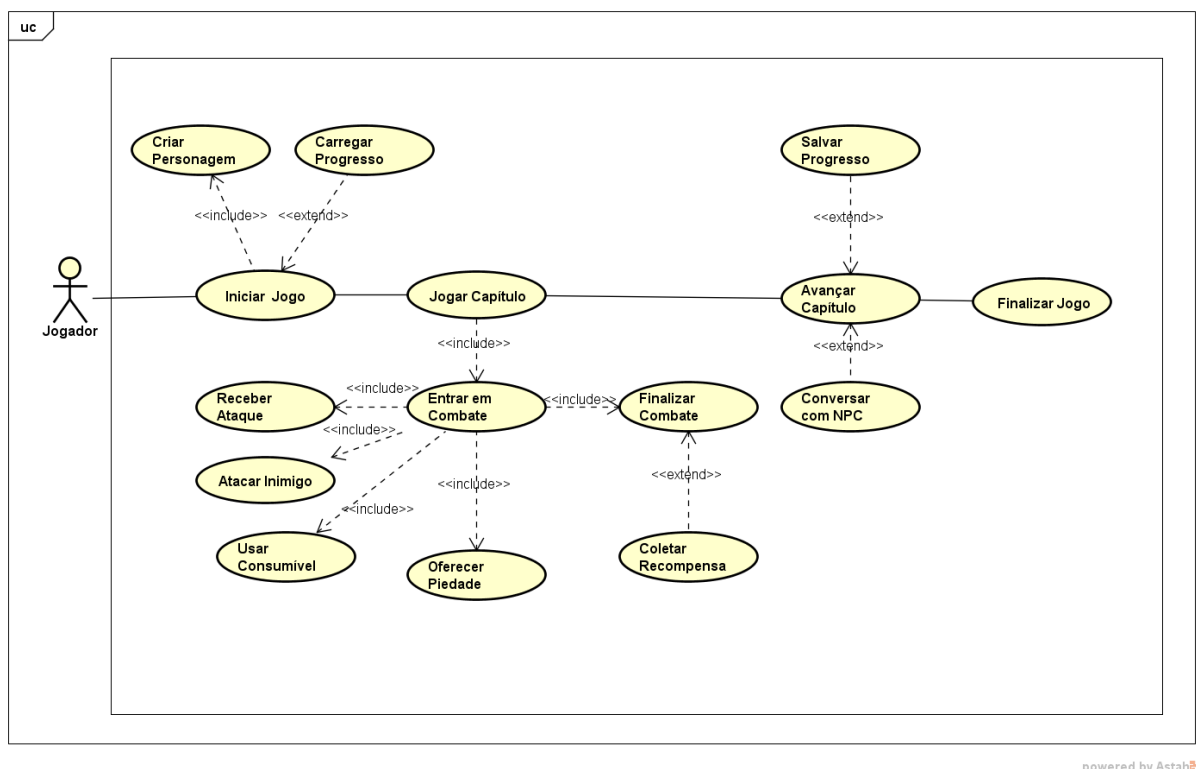


Figura 1: Caso de uso

3.1.1.Caso de Uso: Iniciar Jogo

Descrição: O jogador pode iniciar uma sessão de jogo, podendo carregar um salvamento anterior ou criar um personagem, definindo o nome e gênero deste.

Ator: Jogador.

Cenário Principal:

1. O jogador abre o jogo e acessa a tela inicial.
2. O jogador escolhe a opção **iniciar jogo novo**.
3. O jogador é direcionado a tela de criação de personagem, onde define o nome e gênero.
4. O sistema inicializa os atributos, inventário e posição inicial do jogador no Andar 0 da torre, no Prólogo.

Cenário alternativo:

1. O jogador abre o jogo e acessa a tela inicial.
2. O jogador escolhe a opção **continuar jogo salvo**.
3. O sistema verifica a existência do arquivo de salvamento (*saves/player_save.bin*).
4. O sistema carrega os dados do personagem e o posiciona no andar correspondente ao último ponto salvo.

3.1.2.Caso de Uso: Jogar Capítulo

Descrição: O jogador avança pela narrativa de um andar da torre, interagindo com diálogos e, obrigatoriamente, entrando em combate contra o inimigo presente no andar para poder avançar de andar e de capítulo.

Ator: Jogador.

Cenário principal:

1. O jogador entra no andar correspondente ao capítulo atual.
2. O jogador acompanha a introdução narrativa e os diálogos disponíveis com o inimigo.
3. O jogador entra em combate com o inimigo do capítulo.
4. O jogador vence o combate e avança para o próximo andar.

Cenário alternativo:

1. O jogador entra no andar correspondente ao capítulo atual.
2. O jogador entra em combate com o inimigo do capítulo.
3. O jogador é derrotado em combate.
4. O sistema restaura a vida do jogador ao máximo, salva o progresso e exibe a tela de morte.
5. O jogador retorna ao início do mesmo capítulo.

3.1.3.Caso de Uso: Entrar em combate

Descrição: O jogador participa de uma batalha por turnos contra o inimigo do andar, podendo atacar e causar dano ao inimigo, esquivar dos ataques recebidos, usar consumíveis para recuperar a vida ou oferecer piedade ao oponente.

Ator: Jogador.

Cenário Principal:

1. O jogador entra no loop de esquiva, no qual ocorre os padrões de ataque do inimigo que ele precisa esquivar.
2. O jogador escolhe atacar o inimigo, reduzindo sua vida.
3. O ciclo de ataque e esquiva se repete até a vida do inimigo chegar a zero.

4. O jogador pode escolher uma das armas geradas ao fim do capítulo para equipar, substituindo seu equipamento atual e modificando seus atributos.

Cenário alternativo:

1. O jogador recebe os padrões de ataque do inimigo e precisa esquivar deles.
2. O jogador escolhe oferecer piedade ao invés de atacar.
3. O sistema interpreta a decisão de poupar o inimigo, alterando o karma do jogador e o desfecho narrativo do capítulo.
4. O jogador pode escolher uma das armas geradas ao fim do capítulo para equipar, substituindo seu equipamento atual e modificando seus atributos.

3.1.4.Caso de Uso: Avançar Capítulo

Descrição: Após vencer o combate de um andar, o desce até o ponto de descanso, onde pode salvar seu progresso, conversar com o Vigia ou continuar subindo a torre, ou sair do jogo ao mesmo tempo que o progresso é salvo.

Ator: Jogador.

Cenário Principal:

1. O sistema direciona o jogador ao ponto de salvamento.
2. O jogador escolhe salvar o progresso.
3. O jogador escolhe continuar subindo a torre, avançando ao próximo capítulo.

Cenário alternativo:

1. O sistema direciona o jogador ao ponto de salvamento.
2. O jogador escolhe salvar o progresso.
3. O jogador escolhe conversar com o Vigia.
4. O jogador seleciona uma das opções de diálogo disponíveis para aquele andar, revelando mais informações sobre a história de cada boss e sobre o espaço onde ele está.
5. O jogador retorna ao menu do ponto de salvamento.

3.1.5.Caso de Uso: Finalizar Jogo

Descrição: O jogador termina os capítulos disponíveis e visualiza o desfecho resultante de suas escolhas de matar ou poupar os inimigos que afetam o seu karma.

Ator: Jogador.

Cenário Principal:

1. O jogador assiste o desfecho disponível com base em suas escolhas.

3.1. Requisitos Não-Funcionais

São as especificações e restrições sobre os serviços ou funções providas pelo sistema, como usabilidade, manutenibilidade, confiabilidade, desempenho, portabilidade, entre outros.

- **Escalabilidade**

O sistema deve permitir a adição de novos tipos de ataques, armaduras, armas e capítulos sem reestruturações grandes.

- **Manutenibilidade**

O código deve seguir padrões claros e modularizados para facilitar futuras manutenções.

- **Portabilidade**

Idealmente a aplicação deve ser executável em ambientes Linux ou Windows, ampliando o alcance do jogo.

4. Diagramas do Sistema

O objetivo desta seção é apresentar e explicar os principais diagramas e, desse modo, representar diferentes aspectos do desenvolvimento de software.

4.1. Diagrama de Caso de uso

Um diagrama de caso de uso é uma representação visual que mostra atores, funcionalidades e interações, usada para definir o comportamento e os objetivos do sistema do ponto de vista do usuário.

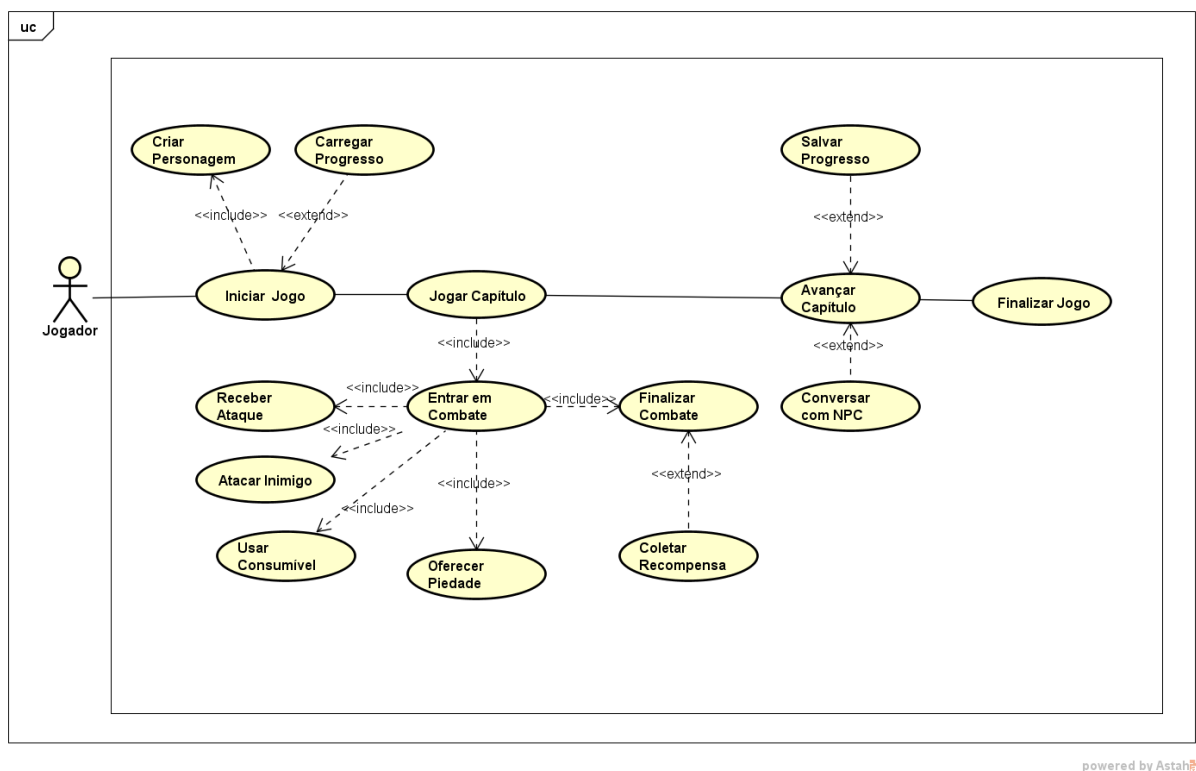


Figura 2: Caso de uso

A Figura 2 apresenta o diagrama de casos de uso que descreve a interação do jogador com o sistema, representando assim de forma mais abstrata as ações e fluxos existentes.

1. Jogador:

- O ator "Jogador" pode realizar o caso de uso "Iniciar Jogo", que inclui a criação de um novo personagem ("Criar Personagem") ou estende-se para o carregamento de uma sessão existente ("Carregar Progresso").
- O "Jogador" realiza o caso de uso "Jogar Capítulo", que inclui obrigatoriamente "Entrar em Combate" contra o inimigo do andar atual.

- ## 4.2. Diagrama de Classe/Structs

[illegible]

A Figura 3 apresenta o diagrama de classes que demonstra as estruturas presentes no sistema, seus atributos, funções e interações, sendo estas: **Player**, **Inimigo**, **Combate**, **AtaqueInimigo**, **Inventario**, **Arma**.

- **Player:**

Esta struct gerencia os dados de progresso do personagem, incluindo atributos como nome, gênero, nível, vida e karma, além de métodos como *mover_player*, *criar_player*, *tomar_dano* e *atacar_inimigo*, que controlam o estado e as ações do jogador durante a partida. Relaciona-se diretamente com *Atributos*, *Inventario* e *Posicao*.

- **Inimigo:**

Esta struct representa os chefes encontrados em cada andar da torre, armazenando vida, sprite, posição e um vetor de *AtaqueInimigo* correspondente aos padrões de ataque daquele combate, além de métodos como *inimigo_criar*, *inimigo_tomar_dano* e *renderizar_nome_estilizado*, responsáveis por instanciar e processar o comportamento do inimigo durante a batalha.

- **Combate:**

Esta struct controla o fluxo de uma batalha entre Player e Inimigo, gerenciando o estágio atual do combate, e a interação do jogador com o ambiente através de funções como *ataque_colidiu*, *spawnar_ataque*, *atualizar_ataque* e *loop_esquiva*, que processam a colisão de ataques, a geração de novos ataques inimigos e a alternância entre as fases de diálogo, esquiva e ação.

- **AtaqueInimigo:**

Esta struct controla os atributos que regem a velocidade de spawn, velocidade, sentido e direção de deslocamento, dano causado, hitbox, estado atual dos ataques utilizados pelos inimigos. Além disso, ela também armazena o sprite referente a cada ataque. Cada ataque é instanciado por meio das funções *criar_ataque* e *criar_ataque_area*.

- **Inventario:**

Esta struct gerencia os itens do jogador, agregando uma *Armadura*, composta por quatro atributos de tipo *ParteArmadura* (capacete, peitoral, grevas e botas), uma *Arma* e um vetor de *Consumivel*, sendo inicializado com a função *init_inventario*, responsável por inicializar o inventário no início da partida.

- **Arma:**

Esta struct representa as informações necessárias para a arma do jogador, incluindo o dano causado, o tipo de dano, assim como os atributos adicionados ao jogador. A estrutura também armazena o nome do arquivo referente ao sprite da arma. As armas são instanciadas com a função *gerar_arma_aleatoria*.

- **Armadura:**

Esta struct agrega diferentes structs ParteArmadura, criando uma interface mais geral e centralizada para o equipamento.

- **ParteArmadura:**

Esta struct representa as informações necessárias para cada peça de armadura equipada pelo jogador, incluindo assim um nome e descrição, assim como os atributos que ela incrementa, como a sorte, força, defesa e vida.

5. Desenvolvimento da aplicação

O objetivo desta seção é apresentar todo o aparato envolvido no processo de desenvolvimento da aplicação.

5.1. Métodos

5.1.1. Modelagem do sistema

Foi realizada a modelagem da arquitetura do sistema utilizando diagramas UML elaborados no software **Astah**. Foram produzidos diagramas de casos de uso e de classes, permitindo visualizar as funcionalidades disponibilizadas ao usuário, bem como a organização das estruturas de dados e dos módulos da aplicação. Essa etapa contribuiu para o planejamento da implementação e para a documentação do projeto.

5.1.2. Implementação

A implementação foi realizada em três módulos principais, visando reduzir o acoplamento entre componentes e facilitar a manutenção do código.:

- Objects

Módulo responsável pela definição das principais estruturas utilizadas pela aplicação, como jogador, inimigo, armas, armaduras. Também centraliza as principais funções relacionadas a interação, criação e modificação dessas estruturas, servindo como base para os demais módulos do sistema.

- Chapters

Módulo responsável pela implementação da narrativa do jogo. Cada capítulo é representado por uma função específica, responsável por controlar diálogos, eventos, transições da história, instanciação dos inimigos e acionamento do sistema de combate, mantendo a lógica narrativa organizada e independente dos demais componentes.

- System

Módulo responsável pelos principais sistemas do jogo, incluindo combate, salvamento e carregamento do progresso. Nesse módulo encontram-se as funções que implementam as mecânicas mais complexas da aplicação, centralizando regras de negócio e reduzindo a duplicação de código entre os diferentes capítulos.

5.1.3. Testes

A aplicação foi testada em ambiente containerizado com **Docker**, garantindo uma configuração padronizada e isolada sem conflitos ou ausências de dependências. A aplicação foi compilada com o compilador **GCC**.

5.1.4. Controle de versão e colaboração

Durante o desenvolvimento, utilizou-se o **GitHub** como repositório central para versionamento do código e acompanhamento do progresso do projeto. Ademais, foi utilizado branches secundárias para atualizações significativas de forma a reduzir conflitos no código, sendo então mescladas posteriormente.

5.1.5. Outros

Foi utilizado um arquivo de configuração para ferramenta *make* ([makefile](#)), de forma a facilitar a compilação e os testes.

5.2. ECHOBOUND

Apresentação do projeto com ilustrações das telas de forma ordenada, conforme a lógica para a navegação do usuário.



Figura 4: Tela do Menu Inicial



Figura 5: Tela Inicial (Créditos)

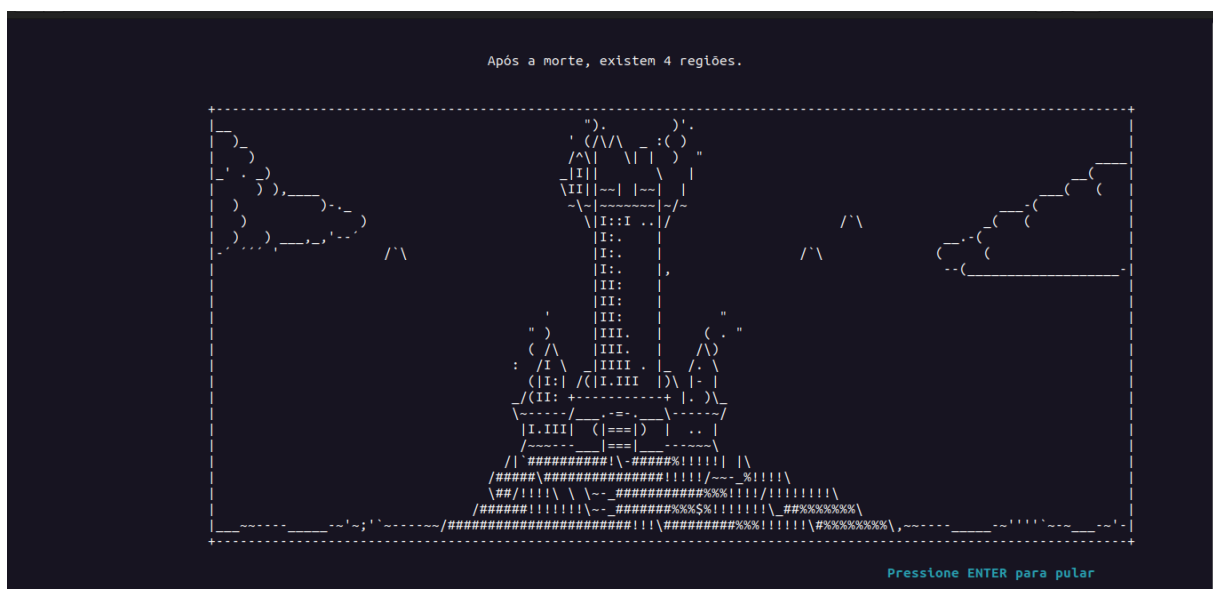


Figura 6: Tela Cutscene Inicial



Figura 7: Tela de Criação de Personagem (masculino)



Figura 8: Tela de Criação de Personagem (feminino)

Ser lembrada.

Klara é uma das inúmeras almas presas no Intervalo.
Um lugar onde memórias desaparecem lentamente. Com o tempo, nomes, rostos e sonhos deixam de existir.
A maioria das almas acaba esquecendo quem era. E então... desaparecem.
Mas algo dentro de Klara ainda resiste.
Mesmo sem se lembrar do próprio passado, sua alma se recusa a desaparecer.
E agora, diante de uma possível saída... Subir a Torre dos Ecos
Klara toma uma decisão...

[illegible]

Figura 10: Tela Apresentação de Boss 1



Figura 11: Tela de Batalha 1

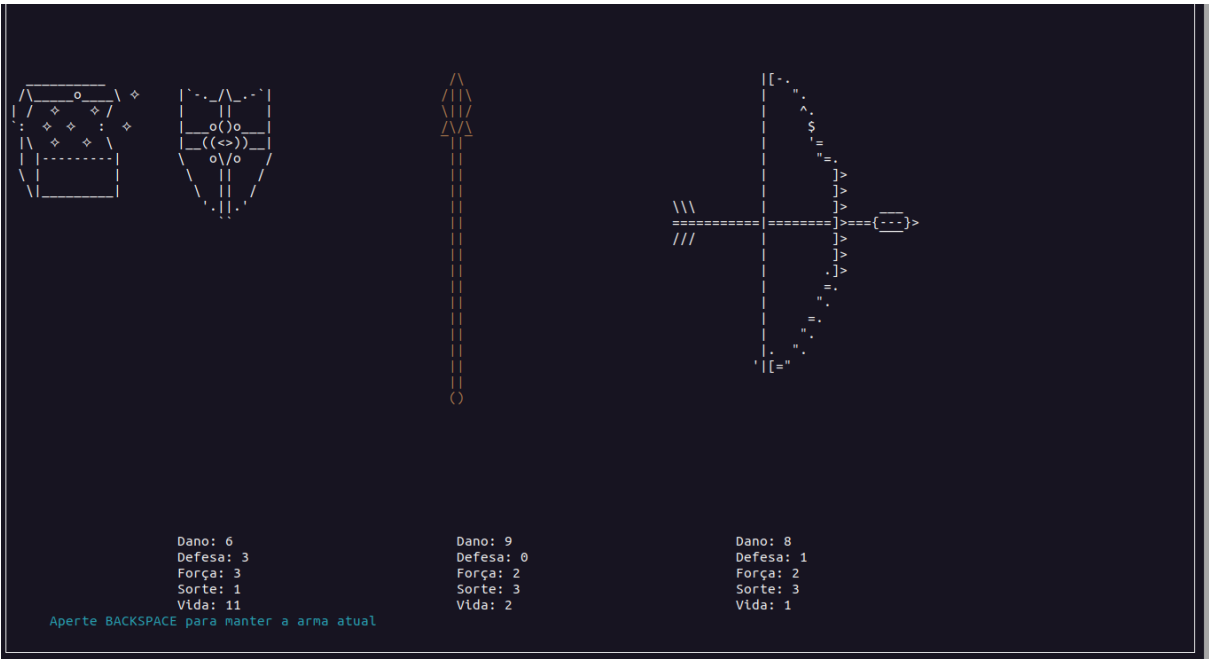


Figura 12: Menu de Loot

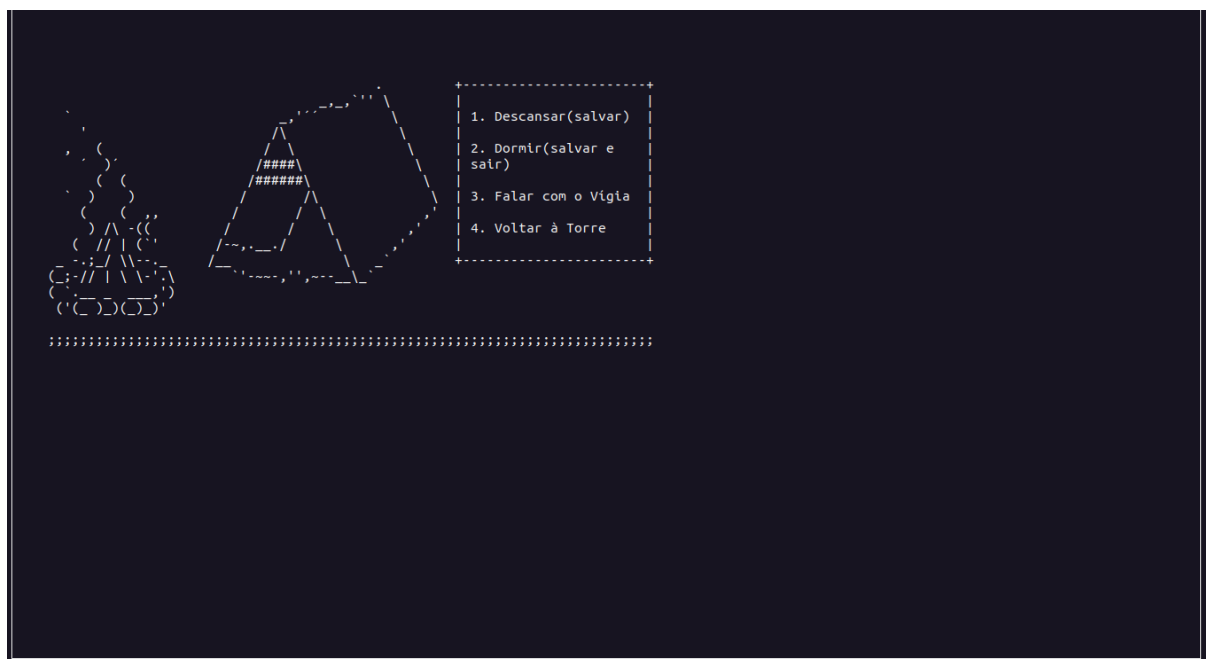


Figura 13: Tela de Descanso



Figura 14: Tela de Combate 2



Figura 15: Tela Apresentação Boss 2



Figura 16: Tela de Combate 3



Figura 17: Tela de Combate 4



Figura 18: Tela de Morte



Figura 19: Tela de Combate 5

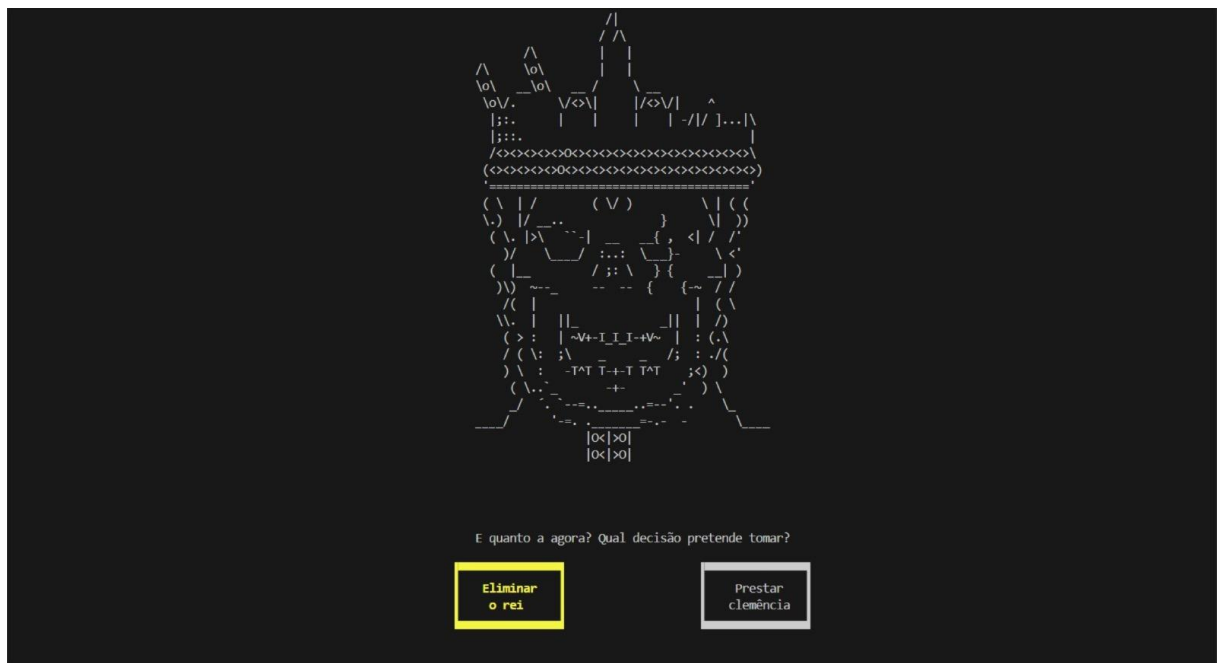


Figura 20: Escolha Final

5.3. Jogabilidade

Nessa seção será descrito as teclas e comandos usados para interação com o jogo.

5.3.1. Comandos de navegação no menu principal

- Mudança de Opção: Keypad (← ↓ ↑ →)
- Confirmar Opção: Enter (↵)

5.3.2. Comandos na tela de criação de personagem

- Inserir Nome: Letras (*abcdefghijklmnopqrstuvwxyz* e suas versões maiúsculas)
- Alternar o gênero: Setas (↓ ↑)
- Confirmar Personagem: Enter (↵)

5.3.3. Comandos de esquiva e controle no combate

- Selecionar Opção no Menu: Dígitos (1234)
- Confirmar Opção: Enter (↵)
- Movimentar: Keypad (← ↓ ↑ →) ou wasd (↑ ← ↓ →)

5.3.4. Comandos de seleção de diálogo e escolha nos menus

- Selecionar Diálogo: Dígito (0123456789)
- Pular Diálogo: Enter (↵)

6. Conclusão

Neste projeto foi desenvolvido um livro-jogo proposto como avaliação na disciplina de Programação Estruturada.

6.1. Limitações

Durante o desenvolvimento do projeto, algumas limitações foram identificadas. Dentre elas, destacam-se:

- I. A ausência de orientação a objetos, característica da linguagem C, resulta em maior acoplamento entre módulos e dificulta a separação de responsabilidades quando comparada a linguagens que oferecem esse paradigma de forma nativa. Contudo, esta limitação também se mostrou como possibilidade de aprendizado, ao exigir implementação manual de estruturas e mecanismos que em linguagem de mais alto nível seriam fornecidos pela linguagem ou por bibliotecas.
- II. A dependência da biblioteca **ncurses** para a interface do jogo pode limitar sua execução em ambientes que não possuam a biblioteca instalada ou que não permitam sua instalação. Para contornar esse problema durante o desenvolvimento, foi utilizada uma imagem Docker configurada com as dependências necessárias.
- III. A persistência de dados foi implementada por meio de arquivos binários contendo estruturas da aplicação. Embora essa abordagem seja eficiente para o contexto do projeto, ela oferece menor flexibilidade para evolução do formato de armazenamento e menor compatibilidade entre diferentes versões da aplicação.

6.2. Perspectiva Futura

Este trabalho não esgota as possibilidades de desenvolvimento da aplicação. Pelo contrário, os resultados obtidos demonstram que o projeto pode servir como base para futuras melhorias e expansões.:

6.2.1. Incremento nos capítulos e na narrativa:

Uma das principais possibilidades de evolução consiste na ampliação da história, com a criação de novos capítulos, personagens, inimigos e eventos. Também podem ser adicionadas decisões que alterem o desenrolar da narrativa, aumentando a rejogabilidade e proporcionando diferentes finais para o jogador.

6.2.2. Manutenção e Evolução do Código:

Como continuidade do projeto, pode ser realizada uma refatoração da arquitetura, reduzindo o acoplamento entre módulos e melhorando a organização do código. Também podem ser implementados sistemas mais flexíveis para gerenciamento de itens, inimigos, capítulos e diálogos, facilitando futuras expansões e a manutenção da aplicação.

6.2.3. Melhorias de desempenho e multithreading:

Embora o desempenho atual seja suficiente para os objetivos do projeto, futuras versões poderão utilizar processamento concorrente para separar tarefas independentes, como reprodução de áudio,

gerenciamento de animações, leitura de entrada do usuário e atualização de elementos da interface. Essa abordagem pode tornar a aplicação mais responsiva e abrir caminho para funcionalidades mais complexas.

7. REFERÊNCIA BIBLIOGRÁFICA E SITES UTILIZADOS

<https://www.asciart.eu/gallery>

<https://ascii.co.uk/art>

<https://patorjk.com/software/taag/>

<https://terminalroot.com.br/ncurses/#15-biblioteca-de-menus>

<https://tldp.org/HOWTO/NCURSES-Programming-HOWTO/>

<https://www.linuxjournal.com/content/creating-adventure-game-terminal-ncurses>

<https://arthrp.gitlab.io/2021/05/15/Games-with-ncurses/>

<https://www.viget.com/articles/game-programming-in-c-with-the-ncurses-library>

ANEXO - ENTREGA DE CÓDIGOS

Os códigos desenvolvidos estão disponibilizados por meio do repositório oficial do projeto no GitHub. O arquivo com extensão *.md* (*markdown*) nomeado *README.md* também apresenta uma breve descrição do trabalho.

Repositório GitHub: <https://github.com/Jogo-pe-UFRRJ-GOTY2026/echobound>